

Unit 7 String in Python

Strings: A sequence of characters is called string. Strings are used by programming languages to manipulate text such as words and sentences. String literals in Python are enclosed by double quotes or single quote. String literals can be span multiple lines. Multiple line strings are enclosed by three double quotes `"""` or three single quotes `'''`.

Example:

```
>>>a="This is single line String"
```

```
>>>print(a)
```

This is single line String

```
>>>a="""this is
```

```
multiple
```

```
line string"""
```

```
>>>print(a)
```

this is

multiple

line string

Assigning string to a variable:

A single line string is assigned to a variable using double quotes like:

```
A="Basit is a good boy"
```

Multiple line string is assigned to a variable using three double quotes like:

```
A="""Basit is a good boy
```

```
He is twenty years old"""
```

Accessing Values in strings:

The characters of a string are accessed by using indexing. The index is written in square brackets []. The first character of string is represented by index 0, second character is represented by index 1 and so on. The last character of a string has index number `length - 1` and it is also accessed by using index number `-1`. It is because Python allows both positive and negative indexing.

Example:

```
a="Good Morning"
```

```
print("The first character is",a[0])
```

```
print("The second character is",a[1])
```

```
print("The last character is",a[11])
```

```
print("The last character is also accessed by a[-1] ",a[-1])
```

output:

The first character is G

The second character is o

The last character is g

The last character is also accessed by a[-1] g

String slicing: To access some part of a string or substring, we use a method called slicing. This can be done by specifying an index range. Slice operation is performed on string with the use of colon (:).

Syntax:

String_name [start : stop]

Example:

```
>>>a="Good Morning"
```

```
>>>print(a[3 : 8])
```

```
    d Mor
```

To display characters from the beginning to a range use [: index]

To display characters from end use [:- index]

To display characters from specific index till the end use [index :]

```
A="ABCDEFGH"
```

```
print(A[: 4])
```

```
ABCD
```

```
print(A[:-3])
```

```
ABCDE
```

```
print(A[2:])
```

```
CDEFGH
```

Traversing a String: Traversing is a technique to iterate the character of string, one character at a time. To iterate over the characters of a string we use for loop as:

```
S="Good Morning"
```

```
for I in S:
```

```
    print(i)
```

String Concatenation Operator (+): String concatenation is the technique of combining two strings. The '+' operator is used for string concatenation. Example:

```
S1='Good'
```

```
S2='Morning'
```

```
S3='Sir'
```

```
A=S1+S2+S3
```

```
print(A)
```

Output: GoodMorningSir

String Replication Operator (*): Python allows us to repeat the given string using repetition operator which is denoted by symbol (*). Asterisk sign (*) when used with one string and one number, it performs string replication operation. e.g.

```
a=3*"Basit"
```

```
print(a)
```

BasitBasitBasit

In operator: This operator is used to check if a value exists in a string or not. It evaluates True if it finds a value in a string and False otherwise. e.g.

```
a="Good Morning"
```

```
"Good" in a
```

True

```
"Bad" in a
```

False

not in operator: This operator is the opposite of 'in' operator. So, if a value does not exist in the string, then it will return a True else it will return a False. e.g.

```
a="Good Morning"
```

```
"Good" not in a
```

False

```
"Bad" not in a
```

True

Built-in String Methods:

len(): This method returns the length of the string.

Syntax len(string)

Example:

```
a=len("Good Morning")
```

```
print("Total Characters:",a)
```

upper(): This method is used to convert lowercase letters in string to uppercase.

Syntax: string.upper()

Example:

```
a="Good Morning"
```

```
b=a.upper()
```

```
print(a)
```

```
print(b)
```

output:

Good Morning

GOOD MORNING

lower(): This method is used to convert uppercase letters in string to lowercase.

Syntax: string.lower()

```
a="Good MORNING"
```

```
b=a.lower()
```

```
print(a)
```

```
print(b)
```

output:

Good MORNING

good morning

replace(): The replace() method replaces a specified phrase with another specified phrase.

Syntax:

```
replace(oldvalue, newvalue, count)
```

Example:

```
a="Today is Tuesday"
```

```
a.replace("Tuesday", "Monday")
```

```
'Today is Monday'
```

Use of escape characters in the String:

Escape character is used to enter character in a string which are illegal. An escape character is a backslash \ followed by the character you want to insert.

Code	Result
\'	Single Quote
\\	Backlash
\n	New Line
\t	Tab

`\b` Backspace
`\r` carriage Return

Example:

```
s = 'It\'s alright.'
```

```
print(s)
```

```
s = "This will insert one \\ (backslash)."
```

```
print(s)
```

```
s = "Good\nMorning!"
```

```
print(s)
```

```
s = "Good\tMorning!"
```

```
print(s)
```

```
s = "You are \"Very Good\" Boy"
```

```
print(s)
```

Output:

It's alright.

This will insert one \ (backslash).

Good

Morning!

Good Morning!

You are "Very Good" Boy